

Reinforcement Learning (24DS742)
Lab Assignment
Task - 1

RL Components Table

RL Component	Student Observation
Agent	The AI player learning to play tic-tac-toe (in this demo, trained by playing against itself/another instance of itself many times)
Environment	The 3×3 tic-tac-toe board, including the opponent's moves and the game rules that determine outcomes
State Representation	The current configuration of the board which of the 9 cells hold an X, an O, or are empty
Action Space	The set of empty cells available at a given turn; the agent chooses one to place its mark in
Reward Function	+1 for winning the game, 0 (or a small/no reward) for a draw, and a negative/zero reward for losing
Learning Approach Used	Value-function-based reinforcement learning the agent estimates $V(s)$, the probability of eventually winning from state s , and updates these estimates through repeated self-play

Answers to Questions:

1. Who does the learning in this environment?

The agent is the AI player (X or O) for whom the move-selection policy is being learnt. It selects an action by the state of the resulting game with the highest estimated value.

2. What is considered as the environment?

Environment returns a new state (board configuration) and a reward associated to that board configuration every time the agent takes an action.

3. What is the game state?

The full 3×3 grid configuration, which is basically a snapshot of which cells will be filled with an X, Ø or nothing at that moment of the game.

4. What can the agent do?

Taking the place in any empty cell (marked with X or O). Game actions are limited as the game goes on.

5. When does the agent get a positive reward?

When it finishes three marks horizontally, vertically or diagonally (i.e. when it wins the game). States that stay closer to a win (such as two-in-a-row) don't get a direct reward, but rather high estimated value for them.

6. What type of Reinforcement Learning do they learn by?

Value-function approach (temporal-difference style updates): The agent stores $V(s)$ and updates the value of states it regularly visits to gradually approach the realized outcome of its self-play games, instead of storing Q-values or deep neural networks.

7. Is the agent trained using labelled data? Justify your answer.

No. No set of (correct move, label) pairs exists. The agent will start playing the game with itself and will update its values based on what it played, whether it was a win, lose or draw: not quite the same as supervised learning.

8. For this case give explanation for the difference between Reinforcement learning and Supervised learning.

In supervised learning you need a set of supervised examples (such as board state X and best move Y) from an expert. In here, the agent is not told what the 'correct move' is. Rather than doing this, it plays a variety of games, watches its results at the end of the game (win/loss/draw) and learns about which states and actions have good payoffs through trial and error and feedback on the rewards (win/loss/draw) instead of knowing them from the start.

Task - 2

Training Episodes observation table

Training Episodes	Games Played	Win %	Loss %	Draw %	Behaviour Observed
100	10	30%	30%	40%	Mostly random play,agent missed obvious blocks
500	14	14%	36%	50%	Actually worse win rate than 100 episodes value estimates still noisy/unstable at this stage.
1,000	10	80%	0%	20%	Sharp jump in win rate; agent stops losing entirely and starts capitalizing on mistakes.
5,000	10	40%	0%	60%	Still never loses, but now forces more draws than win playing more defensively/optimally.
10,000	10	20%	0%	80%	Converges toward drawing almost every game near-optimal play.

9. How does the quality of play change as training episodes increase?

It's not a smooth, monotonic improvement loss rate does drop steadily and hits 0% from 1,000 episodes onward, showing the agent quickly learns to stop making losing mistakes. But win rate is non-monotonic: it peaks at 1,000 episodes (80%) then declines through 5,000 (40%) and 10,000 (20%), with draw rate rising to fill the gap. This reflects the agent shifting from "opportunistically winning when I make mistakes" toward "playing the objectively correct, non-losing strategy" which in tic-tac-toe increasingly means forcing a draw against any reasonable opponent.

10. At approximately how many episodes does the agent begin making intelligent decisions?

Around 1,000 episodes, where loss rate drops to 0% and win rate jumps to 80% a clear inflection point compared to the noisy, losing-prone play at 100–500 episodes.

11. At what stage does learning appear to converge?

By 10,000 episodes, with 80% draws and 0% losses, the agent is playing close to the game-theoretically optimal strategy (tic-tac-toe is a forced draw with perfect play on both sides).

12. Why does win percentage improve with additional training (early on)?

Between 100 and 1,000 episodes, the agent has run enough self-play games to build value estimates that reliably favor blocking threats and creating winning lines, so it starts capitalizing on any mistakes you make rather than playing randomly.

13. Why do most games end in a draw after sufficient training?

Tic-tac-toe has no forced win for either side under optimal play. As training increases past 1,000 episodes, the agent's policy converges toward that optimal, non-losing strategy so instead of winning more, it increasingly neutralizes the game into a draw, which is exactly why your win % drops while draw % climbs at 5,000–10,000 episodes.

14. What happens if training episodes are too small?

At 100–500 episodes, the value function hasn't propagated through enough states yet, so play is inconsistent, occasionally decent, occasionally missing obvious blocks. Notably, your 500-episode run performed worse than 100 episodes, which is a good illustration that learning isn't always smooth this early the value estimates are still noisy and haven't stabilized, so performance can temporarily dip before the agent has seen enough games to average out that noise.

Demo

Train Agent

Number of Episodes

10000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

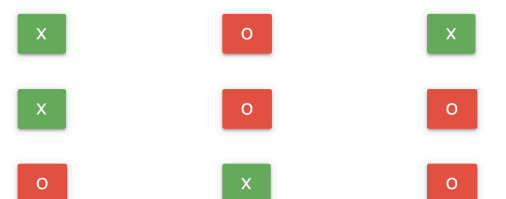
0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 10000)

Draw game. Reset game?



Agent's Value Function

0	0	0
0	0	0
0.0%	0	0

RESET GAME

Demo

Train Agent

Number of Episodes

5000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

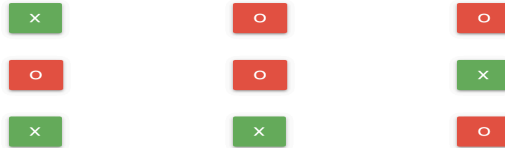
0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 5000)

Draw game. Reset game?



Agent's Value Function

O	O	O
O	O	O
O	O	0.0%

RESET GAME

Demo

Train Agent

Number of Episodes

1000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

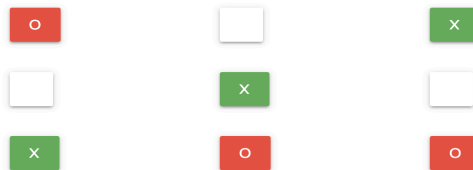
0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 1000)

You won! Reset game?



Agent's Value Function

50.0%	50.0%	50.0%
50.0%	0	50.0%
0	0	0

RESET GAME

Demo

Train Agent

Number of Episodes

500

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

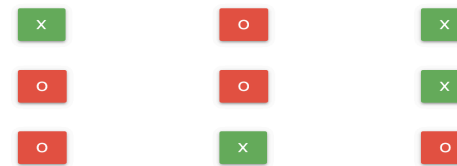
0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 500)

Draw game. Reset game?



Agent's Value Function

0	0	0
0	0	0
0	0	0.0%

RESET GAME

Demo

Train Agent

Number of Episodes

100

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

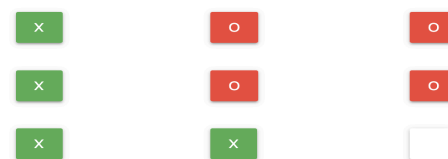
0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 100)

You won! Reset game?



Agent's Value Function

0	0	50.0%
0	0	0
50.0%	0	50.0%

RESET GAME

Conclusion:

The agent was observed to change its behavior, from highly erratic and loss-prone to always non-losing, in 100-10,000 training episodes. The loss rate went all the way to zero after 1000 episodes, indicating a high rate of learning to avoid obvious losses by the agent. To me, it was interesting that the highest win rate occurred at 1,000 episodes (80%), decreasing as more episodes were trained, coming to 80% for draws by 10,000 episodes, indicating the convergence to the optimal strategy for Tic-tac-Toe, which is the strategy a well-trained Tic-tac-Toe opponent will not lose. However, the dip at 500 episodes illustrates that learning via self-play could be

"noisy" in the early stages before stabilizing. Overall, this shows that "reinforcement learning through self-play": The agent does not use any labelled data labeled "correct move", and it updates its value estimates based on the result of playing the games.